ULTRA DRIVE

Game Design Document

Prototype · Hardware-Integrated Racing Project

---

1. Project Overview

Title: Ultra Drive
Genre: Experimental Racing / Hardware-Controlled Prototype
Platform: PC + Physical Hardware
Engine: Unreal Engine 5
Hardware: ESP8266 (NodeMCU), Ultrasonic Sensors (×2), PIR Sensor
Role: Solo Developer
Development Period: 24 April 2025 – 17 May 2025
Project Status: Finished Prototype

Links :

Itch.io : https://old-monkey.itch.io/ultra-drive

---

2. High-Level Concept

Ultra Drive is an experimental racing prototype in which the player controls a vehicle using **real-world physical sensors instead of traditional input devices**. The project explores **stable, real-time communication between custom hardware and Unreal Engine**, translating physical distance and motion into responsive in-game vehicle behavior.

The primary goal is not racing realism, but the investigation of **alternative input systems**, hardware–software integration, and intuitive physical interaction within a game engine.

---

3. Design Goals

1. Achieve reliable, low-latency communication between ESP8266 and Unreal Engine
2. Replace complex control schemes with intuitive, gesture-based physical input
3. Introduce depth in steering through sensor distance rather than binary input
4. Maintain clear, readable gameplay despite unconventional controls

---

4. Core Gameplay Mechanics

4.1 Control Scheme (Hardware-Based)

Action	Input Source	Description
Turn Left	Left Ultrasonic Sensor	Distance-based steering
Turn Right	Right Ultrasonic Sensor	Distance-based steering
Drift	Close-range ultrasonic input	Steering combined with braking
Boost	PIR Sensor	Binary motion trigger

There is **no dedicated acceleration or braking input**. Vehicle behavior is intentionally simplified to keep the player's focus on learning and mastering the experimental control system.

---

5. Steering Depth System

Ultra Drive implements a **distance-based steering depth system**, converting ultrasonic sensor readings into layered vehicle responses.

Sensor Distance	In-Game Response
5–20 cm	Normal steering
2–5 cm	Steering + braking (Drift)

This approach allows low-cost ultrasonic sensors to behave like **analog controls**, increasing expressiveness without additional hardware complexity.

6. Gameplay Modes

6.1 Tutorial Mode

- Free-roaming environment
- No objectives or failure state
- Designed for learning sensor behavior and testing responsiveness

6.2 Endless Mode

- Infinite looping track with ramps
- Emphasis on flow, rhythm, and control mastery
- Difficulty emerges organically from speed and precision

6.3 Classic Mode

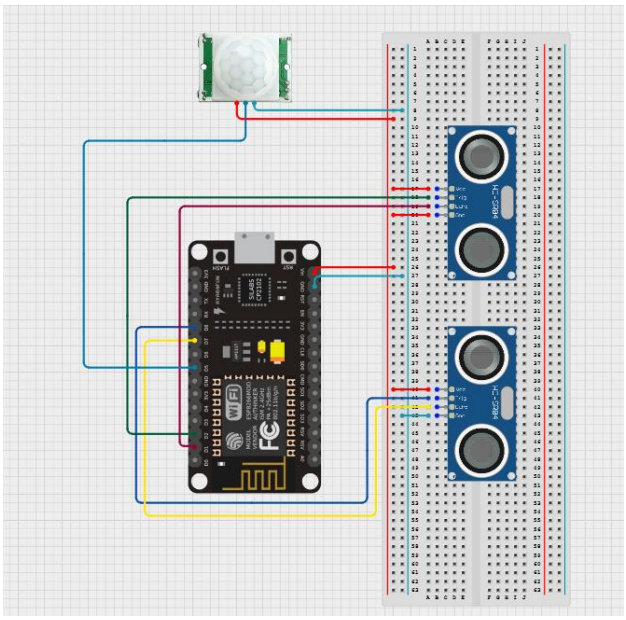
- Fixed track layout
- Player races against time
- Tests consistency and precision with hardware-based controls

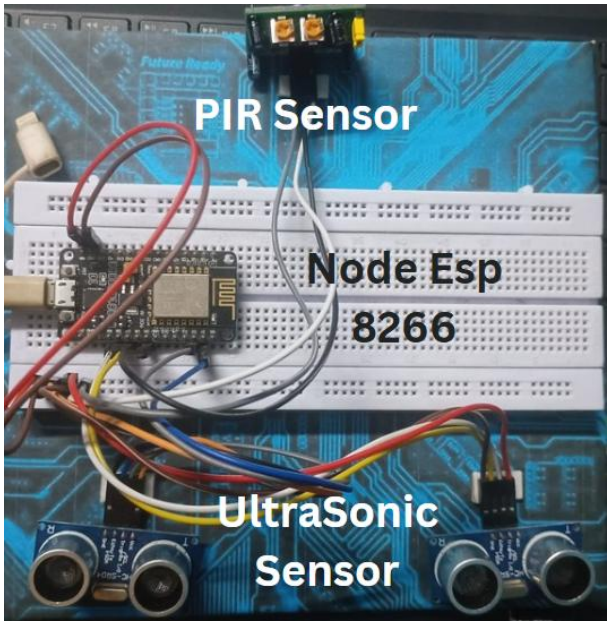
---

7. Hardware & System Architecture

7.1 Hardware Components

- ESP8266 (NodeMCU)
- 2 × Ultrasonic Sensors (Left / Right)
- 1 × PIR Sensor (Boost trigger)





Above figures show circuit diagram of Hardware

7.2 Data Flow Pipeline

Sensors → ESP8266 → Serial Output (Strings)
→ Serial COM Plugin → Unreal Engine
→ String Filtering → Blueprint Events → Vehicle Control

Communication is intentionally **string-based** to prioritize transparency, rapid debugging, and ease of iteration during prototyping.

---

8. Unreal Engine Integration

8.1 Serial Communication

- Implemented using the Serial COM Plugin
- Unreal Engine continuously reads string data from the COM port
- Keywords such as LEFT, RIGHT, LEFTDRIFT, RIGHTDRIFT, and RESET are detected
- Matched keywords trigger Blueprint-based input events that directly control the vehicle

8.2 Input Fallback System

- WASD + Space controls are implemented for debugging and testing purposes
- Hardware input remains the primary and intended control method

---

9. UI Feedback System

A minimal in-game UI displays **left and right hand indicators**, visually representing which sensor input is currently being detected in real time.

This UI serves two key purposes:

- Helps players understand and learn the hardware-based control system
- Assists with debugging, calibration, and validation during development

Visual clarity is prioritized over stylistic complexity to avoid distracting from sensor feedback.

---

10. Technical Challenges & Solutions

10.1 Serial String Formatting Issue

Problem: Input filtering failed due to hidden newline characters (e.g., LEFT\n instead of LEFT).
Solution: All incoming serial strings were trimmed and sanitized before comparison.

Key Lesson: In hardware-driven systems, many bugs originate from invisible data formatting issues rather than logic errors.

10.2 Engine–Plugin Version Mismatch

The Serial COM Plugin officially supported Unreal Engine 5.4, while development was done in Unreal Engine 5.5.

Compatibility was maintained by avoiding engine-level modifications and adapting the project setup to preserve plugin stability.

---

11. Known Limitations

- No progression or meta systems
- Hardware dependency limits accessibility
- Not intended for commercial release

These constraints are **intentional design trade-offs**, aligned with the experimental and exploratory nature of the project.

---

12. Future Improvements

- Steering sensitivity calibration system
- Expanded input visualization and diagnostics UI
- Wireless hardware implementation
- Additional experimental control mappings

---

13. Credits

Game Design, Hardware Integration, Unreal Engine Implementation: Kash

Built using the Unreal Engine vehicle template.
Additional assets sourced from the Unreal Marketplace.

This is a non-commercial experimental prototype developed for learning, experimentation, and technical exploration.